How WRotate Measures Watch Accuracy

The Basic Idea

A mechanical watch ticks at a known rate. A watch rated at 28,800 BPH (beats per hour) should tick exactly 8 times per second. If it ticks slightly faster or slower than that, the watch is gaining or losing time. We measure exactly how much faster or slower, and express it as **seconds per day** (s/day).

For example: if a watch ticks 0.01% too fast, it gains about **+8.6 seconds per day**.

Step by Step

1. Listening to the Watch

The phone's microphone picks up the raw audio of the watch ticking. The audio comes in at 48,000 samples per second (48 kHz) -- that's 48,000 tiny measurements of the sound wave every second.

2. Filtering Out Noise

Watch ticks are high-pitched, sharp clicks. Background noise (fans, traffic, voices) is mostly low-frequency. We run the audio through **high-pass filters** that strip out everything below 4,000 Hz, keeping only the sharp tick sounds. We actually run three filters in parallel at different cutoffs (4 kHz, 6 kHz, 8 kHz) and pick whichever one gives the cleanest signal for each watch.

3. Building an Energy Timeline

We don't need all 48,000 samples per second -- that's overkill for detecting ticks that only happen 8 times per second. So we downsample to about **12,000 samples per second** by taking the peak energy value from every group of 4 raw samples. This creates a compact "energy ring buffer" -- a sliding window of the last 30 seconds of tick energy.

This ring buffer is where tick detection happens. Each position in the buffer represents about **0.083 milliseconds** of time.

4. Figuring Out Which Watch It Is (Auto-BPH Detection)

If the user didn't tell us the BPH, we need to figure it out. We use a technique called **Goertzel analysis** -- it's like asking "how much energy is there at exactly 8 Hz?" (for 28,800 BPH) vs. "how much at 6 Hz?" (for 21,600 BPH), and so on. We check all common BPH values (18,000 / 21,600 / 25,200 / 28,800 / 36,000) and pick the one with the strongest signal, but only if it's decisively stronger than the runner-up.

If the user selected their BPH manually, we skip this step entirely and start detecting ticks right away.

5. Detecting Individual Ticks

Once we know the BPH, we know exactly how far apart ticks should be in the energy buffer. For a 28,800 BPH watch, ticks should be **1,500 ring samples apart** (12,000 Hz / 8 ticks per second).

The engine watches the energy buffer in real time. When it sees an energy spike above a threshold, and the spike is roughly the right distance from the last tick, it registers a tick.

Sub-sample interpolation: The tick might not land exactly on a ring sample boundary. To get more precise timing, we fit a parabola through the three energy values around the peak and calculate where the true peak falls between samples. This gives us fractional-sample precision.

Outlier rejection: If a detected tick is more than 15% off from the expected spacing, it's thrown out -- it's probably not a real tick (maybe a bump or background noise).

6. Pairing Ticks to Cancel Beat Error

This is a subtle but important step. Mechanical watches have a **beat error** -- the tick and the tock aren't perfectly symmetric. One half-swing might be slightly longer than the other. If we measured individual ticks, the beat error would show up as alternating fast/slow readings, adding noise to our rate measurement.

The solution: we **pair consecutive ticks** (tick + tock = one full swing of the balance wheel). A pair always covers one complete oscillation, so the beat error cancels out. We only measure the deviation of pairs, never individual ticks.

For each pair, we calculate:

- **Expected pair interval** = 2 x expected tick interval
- **Actual pair interval** = measured time between the first tick and the tick after next
- **Pair deviation** = the difference, in milliseconds

We keep a running total of pair deviations. This **cumulative pair deviation** is the heart of the measurement -- it's how far the watch has drifted from perfect time since we started listening.

7. The Adaptive Pair Gate

Not all detected pairs are clean. The **adaptive pair gate** decides which pairs to trust:

- We track the recent pair deviations and compute their **MAD** (median absolute deviation) -- a robust measure of how noisy the data is.
- The acceptance threshold = median + 3x MAD, clamped between a floor (0.5 ms) and ceiling (0.58 ms).
- During the first ~2 seconds (**calibration phase**), we use a fixed, loose threshold (0.58 ms) because we don't have enough data yet to compute MAD.
- All pairs are used for deviation tracking and plotting; the gate prevents pathological outliers from corrupting the regression.

8. Computing the Rate (Theil-Sen Regression)

This is where we turn raw tick data into a single number: seconds per day.

We have a list of data points: (elapsed time, cumulative pair deviation). If the watch is running fast, the cumulative deviation grows over time. If it's slow, it shrinks. The **slope** of this line is the rate.

We use **Theil-Sen regression**, which works like this:

1. Take every possible pair of data points
2. Calculate the slope between each pair

3. The **median** of all those slopes is the answer

Why not just draw a best-fit line (ordinary least squares)? Because Theil-Sen is **robust to outliers**. Even if 30% of the data points are garbage, the median slope is still correct. A single bad tick can't pull the line off course.

The slope comes out in milliseconds per second. We multiply by 86.4 to convert to **seconds per day**.

We skip the first 5 pairs from the regression (they're from the calibration phase when the adaptive gate was still learning). We also require at least 10 pairs before showing any rate at all.

9. Stability Detection

We don't just want a rate -- we want to know when it's **settled**. The engine tracks the rate over a 15-second sliding window:

- **Stable** = the rate has stayed within a 3 s/day range for the full window
- **Unstable** = the rate has drifted more than 5 s/day within the window

The different thresholds (3 to gain stability, 5 to lose it) prevent flickering between stable/unstable states. This is called **h ysteresis**.

10. JS Convergence (Auto-Stop)

On the frontend, JavaScript decides when the measurement is "done" and can auto-stop:

1. **Enough time has passed** -- at least 15 seconds if signal is strong, 20 seconds if weak
2. **Enough dots on the chart** -- at least 24 for a 28,800 BPH watch
3. **Bucket rate is stable** -- the last 8 rate snapshots span less than one quantization step (3.0 s/day for 28,800 BPH)

When all three conditions are met, the measurement auto-stops and presents the final rate.

11. What the User Sees

The scatter chart: Each dot represents one tick pair. The X axis is elapsed time, the Y axis is cumulative deviation in milliseconds. If the watch is perfect, dots form a flat line at 0. If it's running fast, they slope upward. The dashed yellow reference line shows the current rate.

The HUD (top-right number): The rate in seconds per day, computed by the Theil-Sen regression. This is the most accurate number we have.

The saved result: When the user taps Save, we store the Theil-Sen rate. This is what shows up on their watch's history chart and in their posts.

Why Dots Look Quantized

The energy ring buffer runs at ~12,000 samples per second. Each tick's position is an integer index in this buffer (plus a fractional offset from interpolation). When we compute pair deviations, they come out as multiples of **0.083 ms** (1 ring sample / 12,000 Hz). Converted to s/day, that's steps of **7.2 s/day**.

Sub-sample interpolation reduces this somewhat, but the dots on the scatter chart still tend to cluster on these bands.

The Theil-Sen regression sees through this quantization -- it computes a smooth rate from the overall trend, not from individual dot positions.

Increasing the ring buffer sample rate (e.g., from 12 kHz to 24 kHz) would halve the quantization step to 3.6 s/day, making dots look smoother. This is a planned improvement for a future iOS app update.

Summary of the Pipeline

```
Microphone (48 kHz raw audio)
    |
    v
High-pass filter (3 parallel: 4/6/8 kHz)
    |
    v
Energy ring buffer (~12 kHz, peak-hold downsampling)
    |
    v
BPH detection (Goertzel analysis -- which watch speed?)
    |
    v
Tick detection (energy threshold + sub-sample interpolation)
    |
    v
Pair grouping (tick + tock -> one pair, cancels beat error)
    |
    v
Adaptive pair gate (MAD-based, rejects outlier pairs)
    |
    v
Cumulative pair deviation (running total of timing drift)
    |
    v
Theil-Sen regression (median of all pairwise slopes -> rate)
    |
    v
Stability detection (3 s/day window for 15 seconds)
    |
    v
JS convergence (auto-stop when ready)
    |
    v
Rate in seconds per day
```